Assignment 4:

Name: Leon Lobo

Admission No: U24AI043

1. Use the city map given in Q1 of assignment 1 and implement the best first search algorithm to reach your destination and compare your results in terms of the number of possible paths explored.

Code:

```python
cities=[
    "Syracuse", "Buffalo", "Detroit", "Cleveland", "Pittsburgh",
    "Columbus", "Indianapolis", "New York", "Philadelphia",
    "Baltimore", "Providence", "Boston", "Portland", "Chicago"
]


graph=[
#  Sy  Bu  De  Cl  Pi  Co  In  NY  Ph  Ba  Pr  Bo  Po  Ch
 [ 0, 150, 0, 0, 0, 0, 0, 254, 253, 0, 0, 312, 0, 0],  # Syracuse
 [150, 0, 256, 189, 215, 0, 0, 0, 0, 0, 0, 0, 0, 0],  # Buffalo
 [ 0, 256, 0, 169, 0, 0, 0, 0, 0, 0, 0, 0, 0, 283],  # Detroit
 [ 0, 189, 169, 0, 134, 144, 0, 0, 0, 0, 0, 0, 0, 345],  # Cleveland
 [ 0, 215, 0, 134, 0, 185, 0, 0, 305, 247, 0, 0, 0, 0],  # Pittsburgh
 [ 0, 0, 0, 144, 185, 0, 176, 0, 0, 0, 0, 0, 0, 0],  # Columbus
 [ 0, 0, 0, 0, 0, 176, 0, 0, 0, 0, 0, 0, 0, 182],  # Indianapolis
 [254, 0, 0, 0, 0, 0, 0, 0, 97, 0, 181, 215, 0, 0],  # New York
 [253, 0, 0, 0, 305, 0, 0, 97, 0, 101, 0, 0, 0, 0],  # Philadelphia
 [ 0, 0, 0, 0, 247, 0, 0, 0, 101, 0, 0, 0, 0, 0],  # Baltimore
 [ 0, 0, 0, 0, 0, 0, 0, 181, 0, 0, 0, 50, 0, 0],  # Providence
 [312, 0, 0, 0, 0, 0, 0, 215, 0, 0, 50, 0, 107, 0],  # Boston
 [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 107, 0, 0],  # Portland
 [ 0, 0, 283, 345, 0, 0, 182, 0, 0, 0, 0, 0, 0, 0]   # Chicago
]


class Node:
    def __init__(self, state, parent=None, cost=0):
        self.state=state
        self.parent=parent
        self.cost=cost

def f(node):
    return node.cost


class pq:
    def __init__(self):
        self.queue=[]

    def push(self, node):
        self.queue.append(node)

    def pop(self):
        best=0
```

```python
class pq:
    def pop(self):
        for i in range(1, len(self.queue)):
            if f(self.queue[i])<f(self.queue[best]):
                best=i
        return self.queue.pop(best)

    def is_empty(self):
        return len(self.queue)==0


def expand(node):
    children=[]
    for j in range(len(cities)):
        if graph[node.state][j]!=0:
            child=Node(
                j,
                node,
                node.cost+graph[node.state][j]
            )
            children.append(child)
    return children


def best_first_search(start, dest):
    start_node=Node(start)
    frontier=pq()
    frontier.push(start_node)
    reached=[None]*len(cities)
    reached[start]=start_node
    global count_node
    count_node=0
    print("Node explored", "Path cost")

    while not frontier.is_empty():
        node=frontier.pop()
        count_node+=1
        print(f"{cities[node.state]:^15} {node.cost:^5}")

        if node.state==dest:
            return node

        for child in expand(node):
            s=child.state
            if reached[s] is None or child.cost<reached[s].cost:
```

```
                frontier.push(child)

        return None


def breadth_fs(start, dest):
    q=[]
    start_node=Node(start)
    q.append(start_node)
    reached=[None]*len(cities)
    reached[start]=start_node
    traversed=0

    while q:
        node=q.pop(0)
        traversed+=1

        if node==dest:
            return traversed

        for child in expand(node):
            s=child.state
            if reached[s] is None or child.cost<reached[s].cost:
                reached[s]=child
                q.append(child)

    return traversed


def depth_fs(start, dest):
    start_node=Node(start)
    reached=[None]*len(cities)
    reached[start]=start_node

    def df_search(node, cost, reached, explored):

        if node.state==dest:
            return explored

        for child in expand(node):
            s=child.state
            if reached[s] is None:
                reached[s]=child
                explored+=1
                return df_search(
```

```python
def depth_fs(start, dest):
    def df_search(node, cost, reached, explored):
                reached[s]=child
                explored+=1
                return df_search(
                    child,
                    cost + graph[node.state][s],
                    reached,
                    explored+1
                )

    return df_search(start_node, 0, reached, 0)


def print_path(node):
    path=[]
    while node:
        path.append(cities[node.state])
        node=node.parent
    path.reverse()
    print(" → ".join(path))


def main():
    start=cities.index("Syracuse")
    dest=cities.index("Chicago")

    result=best_first_search(start, dest)

    if result:
        print("Path found:")
        print_path(result)
        print("Minimmum path cost:", result.cost)
        print("Nodes explored:",count_node)
    else:
        print("No path found")

    print("Breadth first search nodes explored:",breadth_fs(start,dest))
    print("Depth first search nodes explored:",depth_fs(start,dest))


if __name__=='__main__':
    main()
```

Output:

```
Node explored Path cost
   Syracuse       0
    Buffalo     150
 Philadelphia   253
   New York     254
    Boston      312
   Cleveland    339
   Baltimore    354
  Providence    362
  Pittsburgh    365
    Detroit     406
   Portland     419
 Philadelphia   253
   New York     254
    Boston      312
   Cleveland    339
   Baltimore    354
  Providence    362
  Pittsburgh    365
    Detroit     406
   Portland     419
  Providence    435
   Columbus     483
 Philadelphia   253
   New York     254
    Boston      312
   Cleveland    339
   Baltimore    354
  Providence    362
  Pittsburgh    365
    Detroit     406
   Portland     419
  Providence    435
   Columbus     483
 Indianapolis   659
    Chicago     684
Path found:
Syracuse → Buffalo → Cleveland → Chicago
Minimmum path cost: 684
Nodes explored: 15
Breadth first search nodes explored: 17
Depth first search nodes explored: 14
```

In Breadth first search, all adjacent nodes are explored at same time, leading to larger number of nodes explored.
In Depth first search, one path is explored as deep as possible, leading to goal being found quicker and less number of nodes being explored totally but not necessarily most optimal path.

In Best first search, nodes are explored based on path cost, leading to finding the optimal path by traversing through the least number of nodes in a larger graph.
In smaller graphs as the one given, nodes traversed in all three algorithms are comparable.

2. Consider a building floor plan as given in figure. Design a robot guided evacuation planning using the best first search algorithm. Consider the following constraints:

- **Allowed movements up, down, left and right**
- **Walls and rooms block movement**
- **Evacuation planning covers the route from entry to exit**
- **Design an evaluation cost function with proper justification for your algorithm**

Code:

```python
grid=[
    [1, 1, 1, 1, 1, 1 , 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [1, 0, 1, 1, 1, 1, 1, 'E'],
    [1, 0, 1, 1, 1, 1, 1, 1],
    [1, 'S', 1, 1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 1 , 1]
]


start=(4, 1)
goal=(2, 7)


class pq:
    def __init__(self):
        self.queue=[]

    def push(self, node):
        self.queue.append(node)

    def pop(self):
        best=0
        for i in range(1, len(self.queue)):
            if f(self.queue[i], goal)<f(self.queue[best], goal):
                best=i
        return self.queue.pop(best)

    def is_empty(self):
        return len(self.queue)==0


class Node:
    def __init__(self, state, action, parent=None, cost=0):
        self.state=state
        self.parent=parent
        self.action=action
        self.cost=cost


def f(node, goal):
    return node.cost


def expand(problem, node):
    s=node.state
    children=[]

    moves=[(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in moves:
        s0=(s[0]+dx, s[1]+dy)

        if 0<=s0[0]<len(problem) and 0<=s0[1]<len(problem[0]):

```

```python
def expand(problem, node):
            if problem[s0[0]][s0[1]]!=1:
                cost=node.cost+1
                child=Node(s0, (dx,dy), node, cost)
                children.append(child)

    return children


def best_fs(problem, start, goal):

    node=Node(start, None, None, 0)

    frontier=pq()
    frontier.push(node)
    reached={start: node}
    explored_count=0

    while frontier:
        node=frontier.pop()
        explored_count+=1
        print(node.state)

        if node.state==goal:
            return node, explored_count

        for child in expand(problem, node):
            s=child.state

            if s not in reached or child.cost<reached[s].cost:
                reached[s]=child
                frontier.push(child)

    return None, explored_count

def extract_path(node):
    path=[]
    while node:
        path.append(node.state)
        node=node.parent
    return path[::-1]

def main():
    result_node, explored=best_fs(grid, start, goal)

    if result_node:
        path=extract_path(result_node)
        print("Evacuation Path:", path)
        print("Steps Taken:", result_node.cost)
        print("Nodes Explored:", explored)
    else:
        print("Failure")

if __name__=='__main__':
    main()
```

Output:

```
Evacuation Path: [(4, 1), (3, 1), (2, 1), (1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 7)]
Steps Taken: 10
Nodes Explored: 12
```

Evaluation cost function is just the cost of the node explored, i.e. it has an equal possibility to go in any direction since cost to go in any direction is 1 but it stops when reaching a wall or room.